



MedTech
Mediterranean
Institute of Technology



CS325 - Mobile Software Development

Lab 7 :

Introduction to Android Studio & Android Development

Sahar Sarraj

2024-2025

I. Lab Goals

1. Exploring Android Studio & Android SDK
2. Starting a new Android project and getting a first look
3. Explore the project views, project folders and files
4. Setting up an Emulator / Android Virtual Device
5. Explore the activity lifecycle
6. Challenge – Who's that Pokémon ?

II. Review on how to download Android Studio

Download android studio from the official link: <https://developer.android.com/studio>

More detailed steps are included in this link: <https://developer.android.com/studio/install>

III. A look into Android SDK

The Android **SDK** (Software Development Kit) is a comprehensive set of tools, libraries, and documentation essential for developing Android applications.

→To access the SDK settings, navigate to: More Actions > SDK Manager.

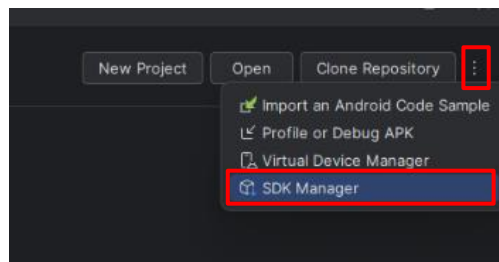


Figure 1: Navigate to the SDK Manager

The **Android SDK** can be broken down into several components:

1. SDK-tools
2. Build-tools
3. Platform-tools
4. The Android Debug Bridge (ADB)

5. Android Emulator

For The Curious:

- **The SDK tools** are arguably the most important parts of the SDK. You will need these tools regardless of the Android version you're targeting. These tools provide the foundation for Android app development and include the complete set of the essential components for creating, testing, and debugging Android applications. Exp: [apkanalyzer](#).
- **The Build tools** were once categorized under the same heading as the Platform tools but have since been decoupled so that they can be updated separately. These tools are used for compiling, linking and packaging the code into a usable or executable form (APK). Exp: [zipalign](#)

-- **APK** stands for Android Application Package and is one of the most repeated terms in any app project. It is an executable file that contains all the data needed to install and run an Android App. And it is unique for each one of them.
- **The Platform tools** are more specifically suited to the version of Android that you want to target. Generally, it is best to install the latest Platform tools, after first installation though, you need to keep your Platform-tools constantly updated. Exp: [adb](#)
- **The Android Debug Bridge (ADB)** is a program that allows you to communicate with any Android device. It relies on Platform-tools in order to understand the Android version that is being used. It is used for managing emulator instances or physical devices, installing APKs, and accessing device logs (via `logcat`).
- **The Android emulator** is what lets you test and monitor apps on a PC, without necessarily needing to have a device available.

IV. New Project & First Look

1. Create a New Project

1. Open your android studio.
2. Click on the button “new project”.
3. Select the template “empty views activity”.
- 4.

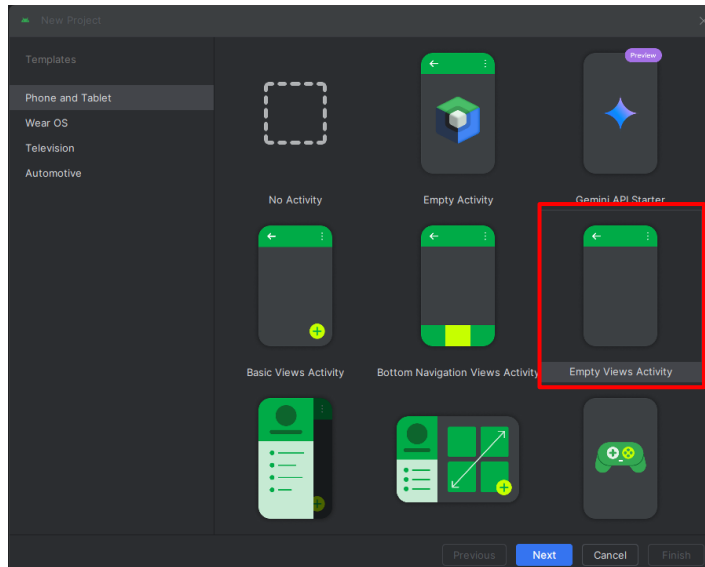


Figure 2: Select the template for your project

1. Give your application a name, such as **My First Android App**
2. Make sure the Language is set to **Kotlin**.
3. Leave the defaults for the other fields.
4. Click Finish.
- 5.

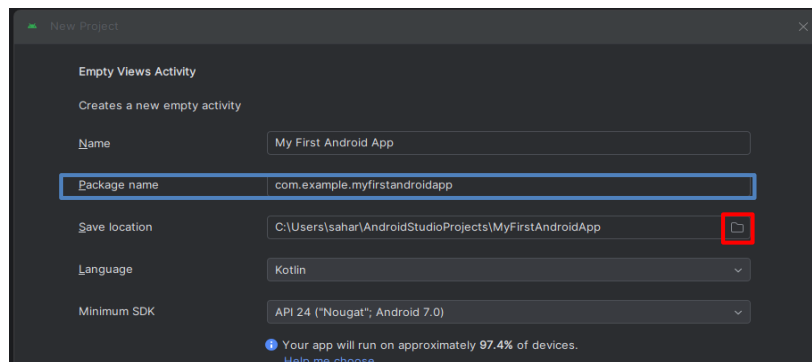


Figure 3: Set up the project.

After clicking 'finish' in Android Studio, the setup process initiates. Please wait while Android Studio prepares the project.

The project opens the file MainActivity.kt automatically and is in Android View (or Project View).

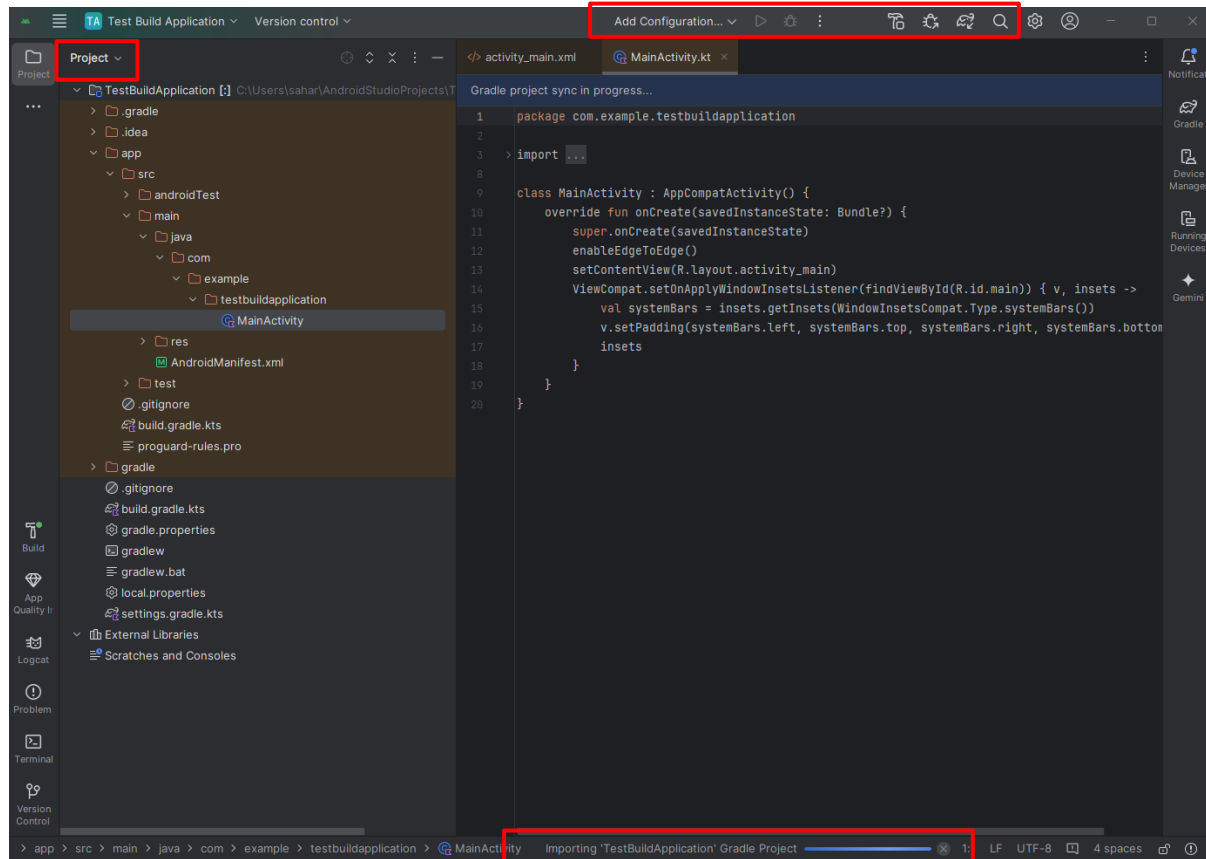


Figure 4: Project is setting up.

2. Project Overview

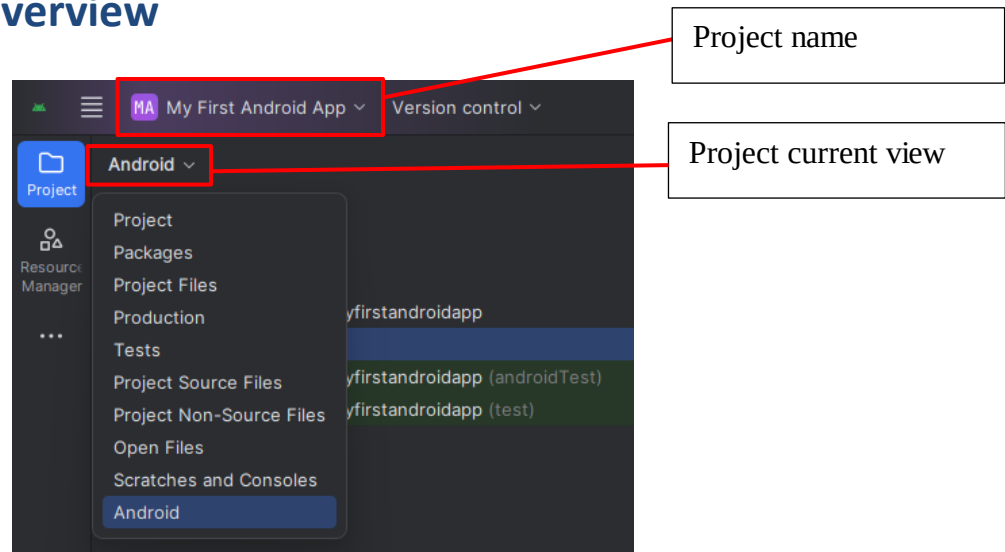


Figure 5: Android Studio Different Views Available

In android studio there are different views to see your project files. The available options are:

1. **Project:** This view shows the actual file structure of the project including source code and external libraries.
2. **Packages:** Organizes project files by package structure, reflecting Java's hierarchical organization.
3. **Project files:** Shows the exact file structure on disk, including hidden files and subdirectories.
4. **Production:** Groups files used specifically for building production APKs.
5. **Tests:** Organizes files related to unit tests and instrumented tests.
6. **Project source files:** Only includes the source folder app.
7. **Project non source files:** Includes non-source files and folders + the app folder.
8. **Open files:** Lists currently open files in the editor.
9. **Scratches and console:** Provides temporary text buffers and displays build outputs and logs.
10. **Android:** This is the standard view that Android Studio sets by default. This view doesn't reflect the actual file hierarchy on disk. Instead, it's organized by modules and file types to simplify navigation between key source files of your project, hiding certain files or directories that are not commonly used. If you look at the files in a file browser, such as Finder or Windows Explorer, the file hierarchy is organized very differently.

Instruction:

- Select the Project view, to see a lot more files and directories.

3. The Project View

The Project View includes the following files and directories:

.gradle folder: This is the toolkit that guides the project-building process of Gradle. It contains all configurations and files used by Gradle to build your project. They are automatically generated.

.idea folder: It is used to store specific project metadata which tells the app how to use the data it describes. It contains configuration files and data for specific functional areas.

app folder: It contains the source code related to the project. Here you write your code, create the user interface, and store assets e.g. images.

- **libs/**
 - It contains private and third-party libraries used within the app.
- **src/**
 - It contains all code and resource files for the module in its subdirectories.
- **androidTest/**
 - It contains code for testing features of the app.
- **test/**
 - It contains the unit testing code, with unit tests executed on the computer.
- **main/**
 - It contains Android code and resources for testing and production deployment.
 - **AndroidManifest.xml**
 - Describes the nature of the application and each of its components.
 - **java/**
 - It contains java and Kotlin related code files.
 - **res/**
 - It contains application resources e.g. image files, font, dimension,

and layout files.

- **local.properties file:** It contains information specific to your local configuration. It is not pushed to GitHub because it contains sensitive information.
- **settings.gradle file:** It handles the various settings for projects and modules. It also displays third part libraries.

4. Make Changes to the App

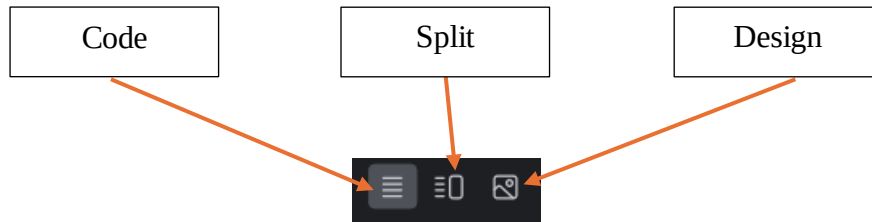
1. Open the file activity_main.xml located as follows:

```
MyFirstAndroidApp > app > src > main > res > layout > activity_main.xml
```

The activity_main.xml is a **layout file** used in Android applications. It defines the user interface for the main activity of an application. The file is written in XML.

There are 3 views of the file activity_main.xml

1. **Code:** This view is where you edit the XML code for your activity.
2. **Split:** This view combines both the "Code" and "Design" views side-by-side.
3. **Design:** This view showcases the visual layout of your activity.



Instruction: Update activity_main.xml

Through the **design view** let's change the activity_main.xml as follows:

1. Change the value of the text property to "My New Text!"
2. Change the background property to #FF9800
3. Change the textcolor property to #3F51B5

Tip: To find a property in the list of all the properties, click on the magnifying glass icon to the right of Attributes, and begin typing the name of the property.

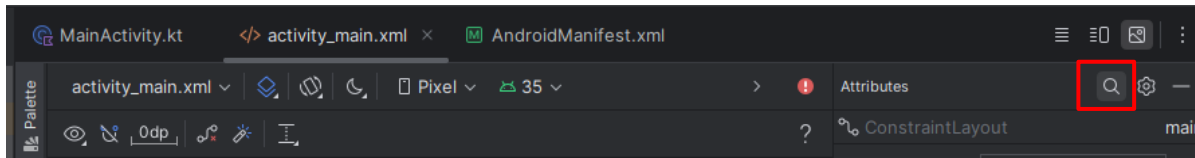


Figure 5: find the magnifying glass in the design view to edit properties

- **What is XML?**

- Stands for “Extensible Markup Language”
- It is used to describe data. Intended to be human readable.
- Used by Android for various resources, such as layouts, storing words, dimensions, colors.
- In Android an xml document is simply made of elements + their attributes

V. Setup an Emulator / Android Virtual Device

It’s important to note the emulator system requirements. If you don’t meet these specs, the emulator might still run but not smoothly. In such cases, consider testing on a physical device instead.

** Note: an AVD is not the same thing as an emulator, it’s actually the configuration that lets you model an actual device by defining hardware and software options to be emulated by the Android Emulator.*

Emulator system requirements ➔

For the best experience, you should use the emulator in Android Studio on a computer with at least the following specs:

- 16 GB RAM
- 64-bit Windows, macOS, Linux, or ChromeOS operating system
- 16 GB disk space

Figure 6: Emulator system requirements.

1. Go to the Device Manager located at your side panel:

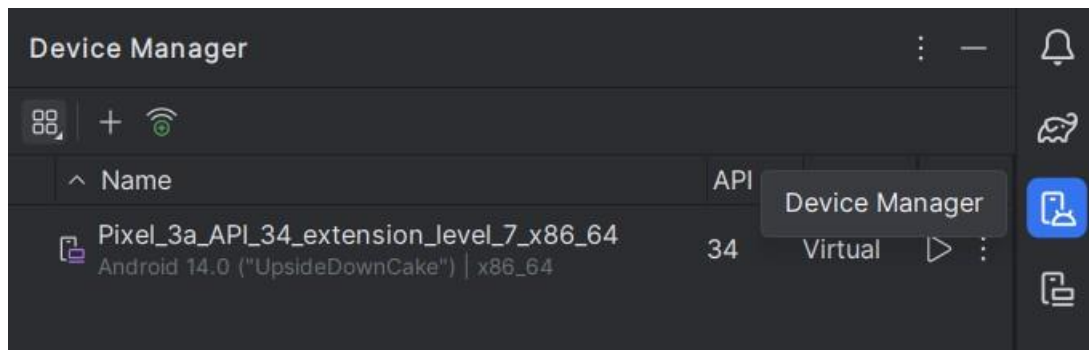


Figure 7: Device Manager in Android Studio.

- In my case I have already created a virtual device. You should click on the '+' sign to create a new virtual device.

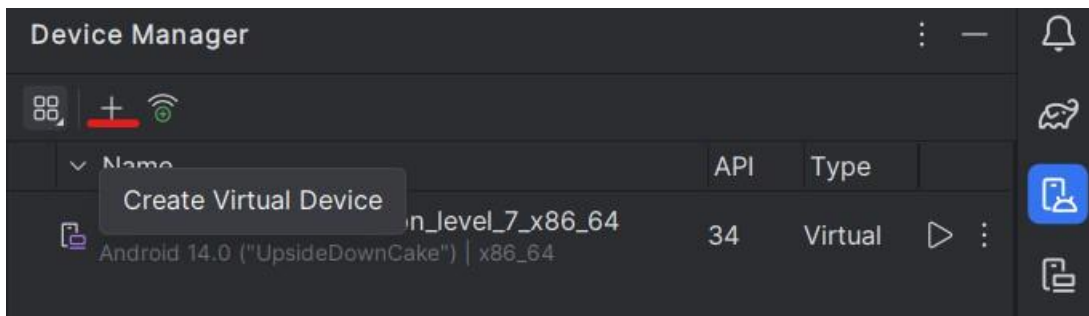


Figure 8: Create a virtual device.

- After clicking on the add device you will get asked about the virtual device configuration

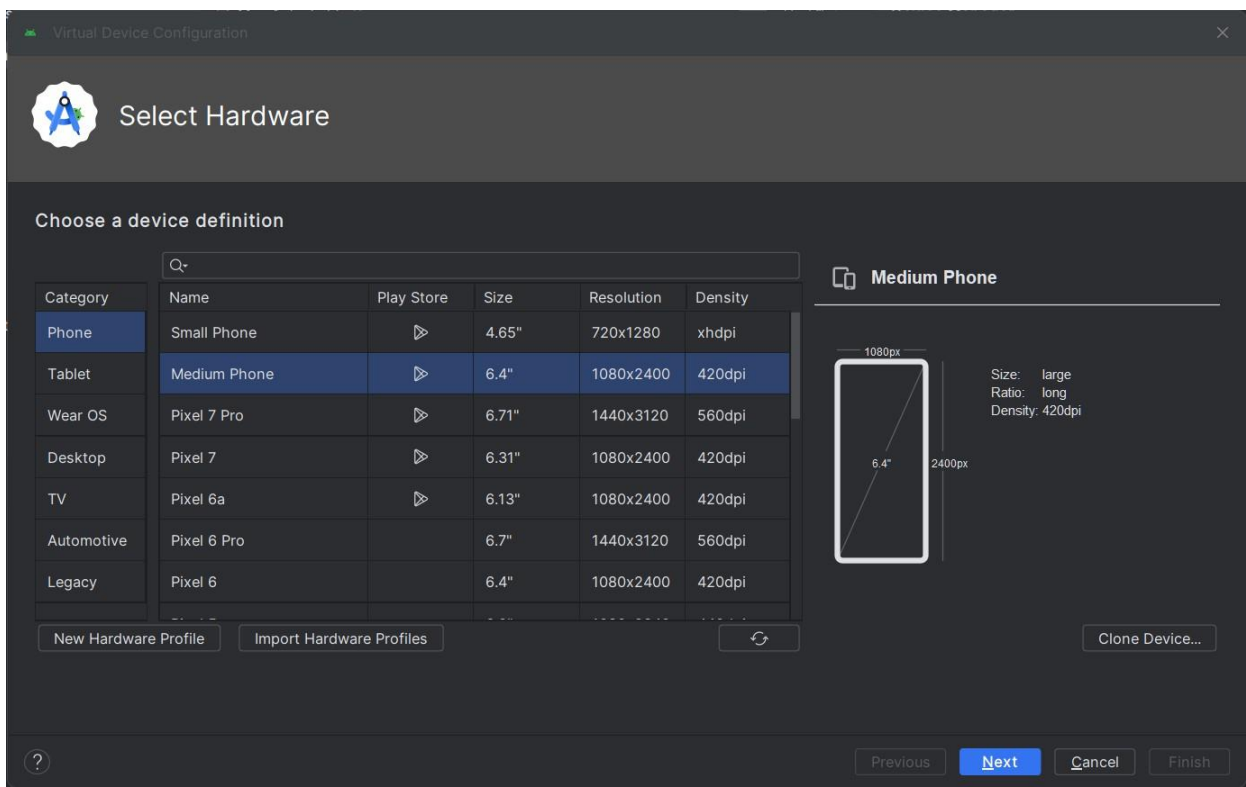
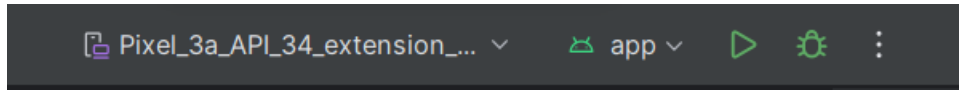


Figure 9: Virtual Device Configuration.

4. Run your app on the emulator:

After you have created an AVD, you can start the Android Emulator and run an app in your project:



Check [this documentation](#) for more info on how to create and manage virtual devices.

VI. Explore the activity lifecycle

1. Go to MainActivity.kt
2. Change the code to this:

```
package com.example.firstapplication

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.util.Log
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }

    override fun onStart() {
        super.onStart()
        print("onStart")
    }

    override fun onResume() {
        super.onResume()
        print("onResume")
    }

    override fun onPause() {
        super.onPause()
        print("onPause")
    }

    override fun onStop() {
        super.onStop()
        print("onStop")
    }

    override fun onRestart() {
        super.onRestart()
        print("onRestart")
    }
}
```

```

    }

    override fun onDestroy() {
        super.onDestroy()
        print("onDestroy")
    }

    private fun print(msg: String){
        Log.d("Activity State ",msg)
    }
}

```

3. Run the application, in the emulator or on your Android device
4. Check the output in Logcat.

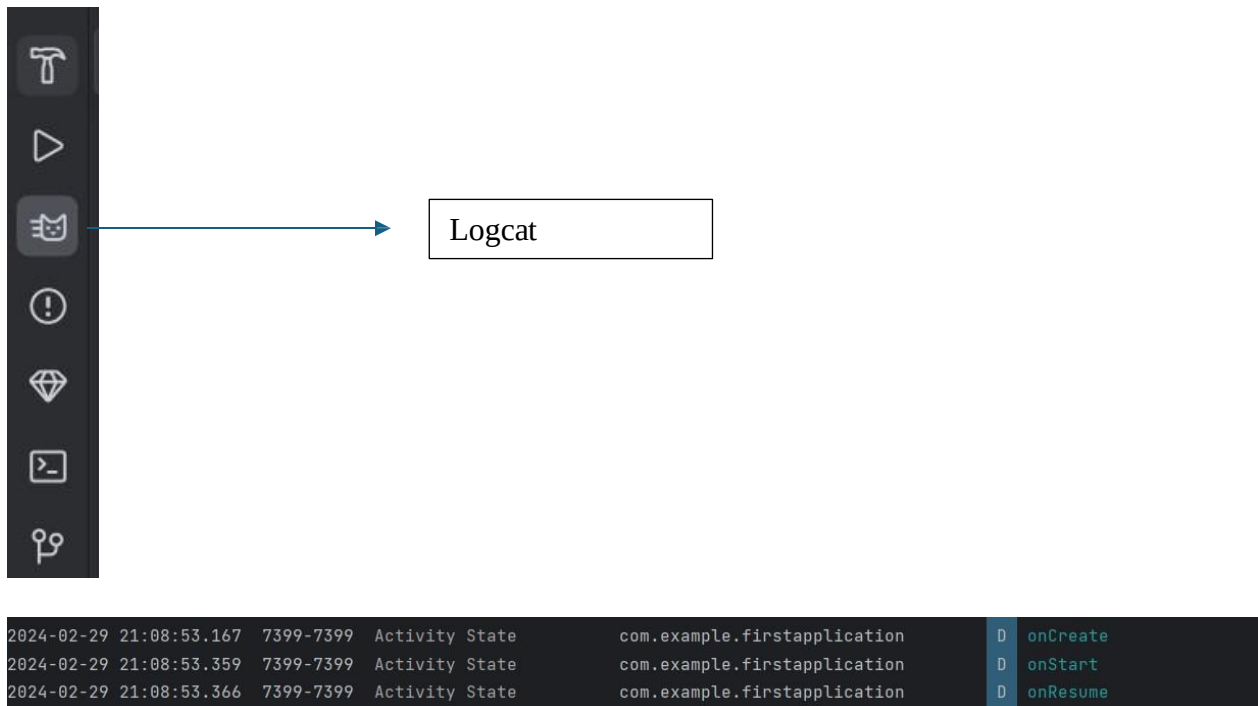
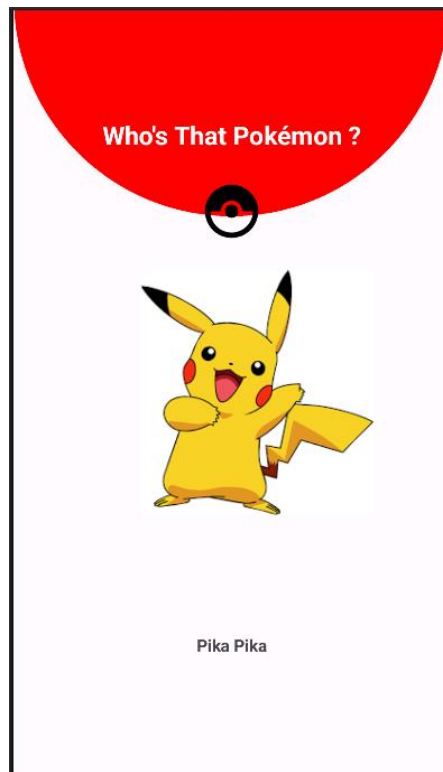


Figure 10: Check the output in Logcat.

VII. The Challenge

- Create a brand new project from scratch (don't forget to close old projects)
- Your final screen should look like this:



Challenge Validation:

To successfully validate this challenge, make sure your app includes:

- **ConstraintLayout:** Make sure that your layout consists of Constraint layout
- **Images:** Ensure that you added the needed image Resources in the Drawables folders and that they occupy the screen space similarly to the given layout.
- **Text Elements:** Style text elements to somewhat match the text attributes of the given layout (style, size, color)
- **Alignment:** Ensure all text and image elements are aligned correctly through the use of the layout_constraint attributes.

Good luck, and have fun!

VIII. Extra resources

- [Android Studio](#) user interfaces.
- [Coding Conventions](#) for a coding style guide for the Kotlin language.
- [Learn Kotlin by Example](#) for a set of small and simple annotated examples.
- [Run apps on the Android Emulator](#) helpful documentation.

6.